

## ScrumBut, but Does It Matter?

### A Mixed-Method Study of the Planning Process of a Multi-Team Scrum Organization

Ville T. Heikkilä, Maria Paasivaara and Casper Lassenius

*Department of Computer Science and Engineering*

*Software Process Research Group*

*Aalto University, Helsinki, Finland*

*e-mail: ville.t.heikkila@aalto.fi, maria.paasivaara@aalto.fi, casper.lassenius@aalto.fi*

**Abstract—Context:** Proponents of the Scrum software development method use the term "ScrumBut" to refer to harmful changes to Scrum. Scrum has been increasingly adopted in large software development organizations. This has led to changes to Scrum practices, but it is not known if these changes are harmful.

**Objective:** We studied how the requirements were planned and managed in the development teams of a large Scrum organization, how well the requirements planning and management practices matched Scrum, and whether the changes were perceived harmful.

**Method:** We quantitatively analysed 435 requirements spanning a time period of approximately one year. We conducted a total of 40 interviews to study the Scrum adoption in the organization and to explain and validate the quantitative results.

**Results:** The main discrepancies between the organization's practices and Scrum were the pacing of the planning process, which was more akin to a continuous process instead of the iteration-paced Scrum model, and the average time it took to develop requirements, which was considerably longer than the time prescribed by Scrum. The latter discrepancy was considered slightly harmful.

**Conclusion:** Changes to the Scrum practices should be evaluated in their context to separate harmful ones from necessary or beneficial changes mandated by the organizational context.

**Keywords—**software engineering, release planning, large-scale agile software development, Scrum

#### I. INTRODUCTION

The Scrum agile software development method [1], [2] originated from a single small team context. Recently, large development organizations have adopted Scrum, which has led to changes to Scrum practices. Proponents of Scrum use the term "ScrumBut" to refer to changes to Scrum which have been made to hide dysfunctions and issues revealed by the unmodified Scrum. One example of ScrumBut given in Scrum.org [3] is "We use Scrum, but we can't build a piece of functionality in a month, so our Sprints are 6 weeks long." ScrumBut in this example is the use of six week long sprints due to the inability of the organization to break down the requirements into small enough user stories. The term implies that the change to the Scrum method was made due to problems in the organization, instead of problems in the Scrum method. Not all changes to the Scrum method are considered ScrumButs. Changing the Scrum practices is

considered acceptable as long as the changes are based on a real need and on a deep understanding about the mechanisms that make Scrum work [1], [4]. The focus of this paper is on the requirements management process and practices of Scrum teams that belong to a large multi-team development organization, which develops a single software system. The goal is the following:

*To study the differences in team-level requirements management practices between the Scrum guidelines and the practice in a large, multi-team Scrum organization.*

Some empirical research describing changes to team-level Scrum planning practices in large, multi-team development organizations exists. Heikkilä et al. [5] described how a multi-team Scrum organization performed release planning by bringing together all members and stakeholders of the development organization for several days. They found that requirements needed to be assigned and prioritized beforehand and that verbal communication was not sufficient to communicate requirements details to teams. The release planning method used was similar to the one described by Leffingwell [6]. Heikkilä et al. [7] performed a case study on how a large Scrum organization performed continuous release planning. The release planning process was characterized as continuous and feature-driven. They found that completely cross-functional generalist teams were not plausible in that environment. Sjøberg et al. [8] conducted a case study where they descriptively compared several development metrics before and after replacing Scrum planning processes with Kanban [9] in a large organization. They found that most of their metrics improved after the change and that the new process was perceived more suitable for their case organization, but they also recognized that the observed changes may have resulted from confounding effects.

The rest of this paper is organized as follows: in Section II, we present an overview of requirements management and planning in Scrum. In Section III we provide our specific research questions and hypotheses, and describe how the qualitative and quantitative data were collected. In Section IV, we present the background, the requirements management practices, and the organization of the development of the case organization. In Section V, we perform the quantitative analysis of the backlog data from the case organization,

compare it with what Scrum prescribes, and try to explain the identified discrepancies. In Section VI, we answer the research questions and discuss the threats to the validity of our results. Finally, in Section VII we provide conclusions and directions for future work.

## II. BACKLOG MANAGEMENT IN SCRUM

Although different sources describing the Scrum method [1], [2] have subtle differences, we can create a synthesis which we shall refer to as the *Scrum backlog management process*. Scrum mandates certain practices. Of these, the sprint planning and the sprint review are the most important in the context of this research. The *Product Owner* is the central role in requirements management in Scrum, as the Product Owner is solely responsible for prioritizing and managing the requirements. The Product Owner also guides the development team in the implementation of the requirements.

The requirements management tool in Scrum is the *product backlog* [1], [2], which contains all the requirements that have been suggested for implementation in the software. The product backlog is kept in priority order at all times so that the more important a requirement is, the closer to the top of the backlog it resides. These *backlog items* are usually written as user stories which contain a short description of who needs the requirement, what the requirement is, and why it is needed. Any stakeholder can propose the addition of a backlog item to the product backlog, but the Product Owner has the sole power to add and prioritize backlog items. Whenever a backlog item is added to the product backlog, the development team together with the Product Owner estimates the size of the item. Based on the size and importance of the new item, the backlog item is then prioritized into the product backlog. The most important items near the top of the backlog should be estimated more precisely, while the items closer to the bottom can be estimated very coarsely.

Different authors in the agile software development literature have proposed different scales for estimating backlog items in Scrum. Most suggest that the requirements should not be estimated using a real measure such as man hours, but instead using abstract numbers [10], [11]. In addition, a non-linear series such as the (approximate) Fibonacci numbers ( $\frac{1}{2}$ , 1, 2, 3, 5, 8, 13, 20) [10], [11] or an exponential series (for example 1, 2, 4, 8, 16) [11] is recommended when abstract numbers are used. The reasoning behind the recommendations for non-linear series is based on intuition: as the estimate grows, it becomes less accurate, and the estimate series should reflect this with increasingly large gaps between the values [10], [11]. Scrum employs a simple metric to measure the progress of development. The development is performed in set length development iterations, or *sprints*. The development rate, or *velocity*, is measured in story-points per development sprint. Scrum does not mandate any other metrics besides the development velocity.

Development sprints have maximum length of four weeks [1], [2]. Tracking the actual time spent on each backlog item or task is not done in Scrum [1], [2]. Instead, the sprint

velocity ( $v_x$ ) of the sprint ( $x$ ) is calculated by summing the size estimates ( $e(n)$ ) of the backlog items ( $n$ ) completed in the sprint. Each sprint begins with a sprint planning meeting. In the meeting, the Product Owner and the developers first re-estimate the top backlog items in the product backlog if needed. The team then selects backlog items from the top of the product backlog so that the sum of the estimates is less or equal to the velocity of the previous sprint (1). The selected backlog items form the sprint backlog of the following sprint.

$$\sum_{i=1}^{n_{max}} e(n_i) \leq v_{x-1} \quad (1)$$

This is called sprint planning part one. Although backlog items have no hard size limit, no backlog item which does not fit into a single sprint can be included in the sprint backlog. Such large items must be split into smaller parts before they can be included. In part two of the sprint planning, the team looks at the backlog items included in the sprint and writes down tasks that need to be performed to implement the items. The tasks are also estimated using man-hours as the unit.

The effort left for completing the backlog items in a sprint is not tracked during the sprint. Instead, the effort left in each task is kept up to date. The estimate sum of the backlog item included in a sprint thus stays constant, except when a backlog item is removed due to overcommitment or added due to slack. The number of backlog items under development in each sprint is not directly dependent of the sum of the effort estimates of the backlog items and is limited between 1 and  $v_x$ . In other words, a sprint backlog, in theory, could include only a single backlog item with an estimate equal to  $v_x$ , or  $v_x$  backlog items with effort estimate of one. Typically, a sprint backlog contains several backlog items with different estimates.

Scrum does not expect the estimates of the backlog items to be accurate [1], [2]. In the best case, the sprint backlog is implemented during the sprint with little or no slack. If the user stories were overestimated, there is too much slack at the end of the sprint. The team and the Product Owner can decide to add additional user stories to the sprint to fill the slack, or the slack can be used to perform general system improvements, learning, or other valuable work that is not directly related to the backlog items. If the backlog items were underestimated, all backlog items cannot be completed during the sprint. The developers and the Product Owner have to decide which backlog items to remove from the sprint backlog. This should be performed immediately when the overcommitment is noticed to let the developers concentrate on the items that can be realistically completed during the sprint.

When a sprint ends, the developers demonstrate the backlog items they have completed during the sprint in a Scrum practice called the sprint review. In the sprint review, the Product Owner officially accepts the backlog items as completed, at which point they can be removed from the product backlog (i.e. closed). It is also possible that the

implementation is not sufficient and the development of a backlog item needs to be carried over to the following sprint.

Following the Scrum process, the time a backlog item is under development before being accepted by the Product Owner is quantified by the sprint length ( $l$ ) [1], [2]. Independent of their estimate, there is large probability ( $s_1$ ) that a user story is completed in a single sprint, meaning that the median time under development ( $\tilde{t}$ ) for each backlog item is equal to  $l$ . There is a small probability ( $s_2$ ) that a user story is completed in two sprints, an even smaller probability ( $s_3$ ) that it takes three sprints, and so on. Thus, the expected (mean) value of time under development ( $\bar{t}$ ) becomes (2). The values of  $s_i$  depend on the individual team and need to be empirically gathered. To summarize, according to the Scrum literature [1], [2]  $s_i$  should very quickly approach 0 as  $i$  grows and  $s_i \gg s_{i+1}$  for all values of  $i$ .

$$\bar{t} = l \sum_{i=1}^{\infty} i s_i \quad (2)$$

Release planning in Scrum is performed by the Product Owner either based on the release date, or the release contents [1], [2]. In both cases, the Product Owner selects the top ( $n$ ) backlog items from the product backlog as the release backlog.

In date-based release planning (3), the Product Owner multiplies the expected sprint velocity ( $v_{exp}$ ), which the Product Owner has estimated, with the number of sprints ( $m_1$ ) before the release date to calculate the release capacity. Scrum does not provide specific guidelines for estimating the expected sprint velocity [1], [2]. The Product Owner then includes backlog items starting from the top of the product backlog ( $n_1$ ) until the release capacity is full.

$$\sum_{i=1}^{n_{max}} e(n_i) \leq m_1 v_{exp} \quad (3)$$

In content-based release planning (4), the Product Owner first selects the backlog items which need to be included in the release ( $n_1 \dots n_{rel}$ ) from the top of the backlog. The Product Owner then sums up the estimates of the backlog items and divides the result by the expected sprint velocity to calculate the expected number of sprints ( $m_2$ ) needed to complete the release. The Product Owner can change the contents of the release backlog at any time as long as the backlog items in the product backlog have not yet been included in a development sprint.

$$m_2 = \lceil \frac{\sum_{i=1}^{rel} e(n_i)}{v_{exp}} \rceil \quad (4)$$

### III. METHODOLOGY

#### A. Research Questions and Hypotheses

In this paper, we aim to reach our research objective by answering the following research questions:

RQ1 How were user stories planned and managed in the teams of the case organization?

RQ2 Did the user story planning and management practices in the teams differ from the Scrum practices, and if they did, how were the differences explained?

To answer RQ1, we descriptively analyse quantitative and qualitative data from the case organization. In the Scrum process, user stories are mostly set in progress in a sprint planning meeting and closed in the following sprint review meeting. The first corollary of this practice is that the mean time user stories are under development is approximately equal to the sprint length. The second corollary is that the estimates and the time in development for user stories are independent. This contradicts intuition, which is that the size of a user story has a positive correlation with development time, i.e. larger requirements take longer time to complete. To answer RQ2, we first formulate the following three hypotheses and analyse the quantitative data to test them:

- $H_0^1$  User stories are taken into development during the sprint planning day and closed during the sprint review day.
- $H_1^1$  User stories are taken into development and closed also on days other than the sprint planning day and the sprint review day.
- $H_0^2$  The median time under development is equal to the sprint length ( $\tilde{t} = l$ ) and the mean time under development is approximately equal to the sprint length ( $\bar{t} \approx l$ ).
- $H_1^2$  The  $\tilde{t}$  and  $\bar{t}$  are significantly different from  $l$ .
- $H_0^3$  The estimate and the time under development of user stories do not correlate.
- $H_1^3$  The time under development and the estimates of user stories have a positive correlation.

Following the quantitative analysis, we use qualitative data to validate the results and to explain any identified differences compared to expectations set by the Scrum process. In the rest of this section, we describe the collection of the quantitative and qualitative data, and the analysis of the qualitative data.

#### B. Quantitative Backlog Data Collection

The user story data was collected from the backlog management tool of the case company. The data set contained user stories which had been set *in progress*, *closed* or both between August 2011 and November 2012. The user stories had been set in progress when the implementation of the user story begun and closed when the user story was completed. The backlog contained the following fields for each user story although not all values were available for all user stories: the date set in progress, the date closed, one or more responsible teams, and the size estimate.

In addition to the user story data, the following information on each team was received from the case company: the type of work, which was either development, testing or other, and site, which was one of two possible sites. Only user stories that were assigned to a development team or teams were included in the data set, since the other types of teams had different ways of working and did not use Scrum. Two user stories had a development team assigned to it in addition to other, non-development teams. These user stories were excluded from the data set, as it was not possible to deduce how the

Table I  
USER STORIES AND VALID FIELD VALUES

| # of user stories | # of user stories with a valid value in the field |             |        |     |     |
|-------------------|---------------------------------------------------|-------------|--------|-----|-----|
|                   | Estimate                                          | In progress | Closed | DIP | All |
| 435               | 420                                               | 393         | 380    | 370 | 358 |

work had been divided between the teams. The rest of the user stories had a single development team assigned to them. A *days in progress* value (hereafter abbreviated as DIP) was also calculated for each user story by subtracting the closing date of the user story from the date set in progress. Table I shows the following frequencies of user stories included in the data set: The total number of user stories, the number of user stories with a valid estimate, a valid in progress date, a valid closed date, a valid DIP value and the number of user stories with valid values in all the fields.

#### C. Qualitative Interview Data Collection and Analysis

In 2011, we conducted 39 semi-structured [12] interviews, each lasting one to three hours. The goal of these interviews was to gain an overall picture of the adoption of Scrum in the organization. The topics covered in these interviews included, but were not limited to, the organizational history, the requirements management process, the communications processes, the roles and responsibilities in the new process, and the perceived benefits and issues of the new process. The interviewees were selected from all organizational layers and from different teams to collect a diverse and comprehensive set of qualitative data. All interviews were recorded and transcribed by a professional transcription company. The interview transcripts were imported into the qualitative analysis tool Atlas.ti. The interviews were coded and analysed to build an in-depth picture of the requirements management and development process in the case organization. In addition, one additional pair-interview was conducted in 2013. This interview was also recorded and transcribed. The goal of the pair-interview was to validate the results of the analyses which were based on the backlog data and on the 2011 interviews. Both of the 2013 interviewees had extensive experience in the case organization. The first interviewee had been a Product Owner during the quantitative analysis time period, and the second was a general process improvement manager, who had extensive knowledge of the organization and processes. During the interview, the interviewees were shown results from the descriptive and statistical analyses and asked to explain and reflect on the results.

### IV. THE CASE ORGANIZATION

#### A. Background

The case organization had been developing a telecommunications software system for over ten years. They had begun their transformation from a waterfall-like plan-driven process to an agile process in 2009. By 2011, the whole development organization had adopted a Scrum-based development process. The development was performed on two different sites located

in two different European countries. During the years 2011 and 2012 approximately 40 different teams were involved in the development of the software system. This number includes integration teams, testing teams and tool support teams, in addition to the development teams.

#### B. Requirements Hierarchy

The case organization used a four-level hierarchy of backlog items to manage requirements. The highest abstraction level work items were *features*. Features described on a high level what needed to be developed, how much value it would provide and how much it would cost. *Epics* were created based on the features. Each epic described more concretely a part of a feature which could be developed mostly independent of the other epics. Epics were split into *user stories* which described more concretely what needed to be implemented to realize the epic. Development *tasks* were created based on the user stories. They described what needed to be done to implement the user story. Tasks unrelated to user stories were also used to describe problem solving, refactoring, infrastructure improvements and other similar work.

#### C. Organization of the Development

The Product Owners formed their own team which was led by a Chief Product Owner. The Product Owner team was jointly responsible for managing the development of the software. The long-term development was planned by a product manager. The development teams in the case organization consisted of five to seven developers in each team. The Product Owners were not tied to the teams. Instead, one or more Product Owners and teams formed a temporary sub-organization around each epic which was taken under development. The number of teams and Product Owners in the sub-organization depended on the size of the epic and the state of its development. The organization of the development in the case organization could be characterized as epic-driven.

The product manager was mainly responsible for the creation and management of features although the Product Owners and the development teams could assist her with the technical planning. Features were split into epics by the product manager and Product Owners. Again, development teams could assist them with the technical planning. Epics were split into user stories by the Product Owners and development teams. Development teams created development tasks based on the user stories. Figure 1 illustrates the responsibilities of the different stakeholders.

#### D. Backlog Management Process

Features were created by the product managers. Features were prioritized and epics were created in weekly portfolio management meetings participated in by the product manager and Product Owners. New features and epics were added to the product backlog after these meetings. The Chief Product Owner was responsible for keeping the product backlog in priority order. Epics, assigned to one or more teams and

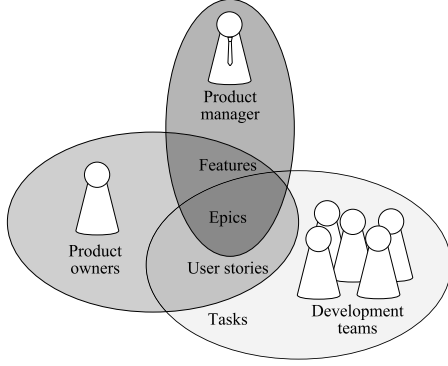


Figure 1. Requirements abstraction levels and management responsibilities

Product Owners, were split into user stories and estimated in bi-weekly backlog grooming meetings. The meetings were participated in by the Product Owner(s) and the team(s) responsible for the development of the epic. The goal was to have at least two sprints worth of user stories in the product backlog for all teams. Each release project was three months long. The contents of each release were based on what epics were completed in time although each epic had a tentative target release.

The teams employed the approximate Fibonacci scale (1, 2, 3, 5, 8, 13, 20) when estimating the effort of user stories. Soon after the Scrum transformation begun in 2009, the managers had noticed that different teams had widely varying scales for estimating user stories. The same story could be estimated as two by one team and as twenty by another. To enable release-level monitoring of progress, the teams were given reference user stories which had a set, organization wide estimate. All teams were then expected to use these stories as a point of comparison when estimating user stories. In 2013, the teams did not use these reference user stories anymore, but the Product Owners expected that the estimates of different teams would still be somewhat similar.

Sprint planning meetings were held bi-weekly, alternating with the backlog grooming sessions. The sprint planning meetings were split into two parts. The team(s) developing an epic and the Product Owner(s) responsible for the epic participated in the first part, where each team selected user stories which they would be implementing during the next sprint. The second part was conducted by each team separately. In the second part, the team(s) split the user stories into concrete development tasks.

## V. RESULTS

### A. Initial Analysis of the Quantitative Data

Before performing statistical analyses on the backlog data, we inspect the data to identify issues that would negatively affect the results of the analyses. As the days in progress (DIP) is measured on the ratio scale, we will use the arithmetic mean as the measure of the central tendency of the DIP. However, the arithmetic mean is not robust towards outliers in the data. Therefore, we inspect the data to see

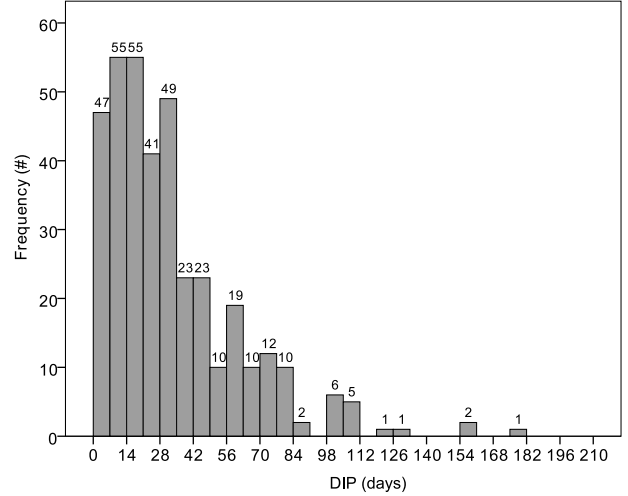


Figure 2. DIP of the development teams' user stories

Table II  
NUMBER OF USER STORIES PER ESTIMATE VALUE

|                   | Estimate (story points) <sup>a</sup> |    |    |    |   |    |    |    |    |    |    |    |
|-------------------|--------------------------------------|----|----|----|---|----|----|----|----|----|----|----|
|                   | N/A                                  | 1  | 2  | 3  | 4 | 5  | 8  | 10 | 13 | 16 | 20 | 40 |
| # of user stories | 15                                   | 48 | 27 | 63 | 2 | 84 | 69 | 3  | 50 | 1  | 11 | 2  |

<sup>a</sup>**Bold** indicates valid values

if it contains outliers [13]. Fig. 2 shows the histogram of the DIP of all development teams' user stories. The DIP values are binned in half sprint (7 days) bins. Based on the histogram, we can assume that the user stories that have DIP value equal to or greater than 154 are outliers. This selects three user stories with the DIP values of 154, 157 and 175, and estimates of 2, 16 and 13, respectively. We assume that those user stories, for some reason, were not under active development regardless of their in progress status and exclude them from the data in the following analyses.

Table II shows the number of valid user stories for each estimate value. We can clearly see that the estimates mostly fall into the approximate Fibonacci number sequence categories of 1, 2, 3, 5, 8, 13, 20. In addition, there are a few user stories with an intermediate estimate value and two user stories with the estimate of 40. According to the interviews, the intermediate values should not have been used and user stories with an estimate over 20 should have been split. These user stories are clearly special cases and we will exclude these user stories from the data in the further analyses. In addition, there are 15 user stories with no estimate, which we also exclude. This selects 15 items with no estimate, two items with estimate 4, one item with estimate 7, four items with estimate 10, one item with estimate 16 and two items with estimate 40. After excluding the outliers and the special cases, we are left with 413 valid user stories, of which 348 have a DIP value.

In the data set, different teams have a different number

of valid user stories from different lengths of time. Some teams were created and started working during the study period and others were disbanded. Some teams concentrated on work that was not captured as user stories, such as systems improvements and issue fixing. As such teams present incomplete data, we need to exclude them from the team-level analysis. Based on the mean DIP value of approximately 31, will include only teams that completed at least one user story every other sprint, on average, and that have data from at least twelve sprints (i.e. 168 days). Table III shows the team name, the number of valid user stories and the number of days from which data is available for each included development team. The team names have been obfuscated for confidentiality reasons.

### B. Setting User Stories In Progress and Closing User Stories

According to the interviews, the sprint length was two weeks, and sprint plannings and sprint reviews were conducted on the Wednesdays of odd numbered weeks. Figure 3 shows the distribution of the user stories set in progress (a) and closed (b) over the weekdays of odd and even numbered weeks. The number of user stories that were set in progress or closed on the Wednesdays of odd numbered weeks is considerably more than on any other weekday of either odd or even numbered week. This implies that most often user stories were set in progress during the bi-weekly sprint planning day (105 user stories) and closed during the bi-weekly sprint review day (117 user stories). However, compared with the total number of user stories set in progress (350), less than a third (30%) of the user stories were set in progress during the sprint planning day and compared with the total number of user stories closed (363), approximately a third (32%) of the user stories were closed during the sprint review day.

When we compare these results with the Scrum guidelines, where user stories are very rarely set in progress during days other than the sprint planning day and never closed on days other than the sprint review day [1], [2], it is clear that we must reject the null hypothesis  $H_0^1$ : Backlog items are set in progress during the sprint planning day and closed during the sprint review day, and accept the alternate hypothesis  $H_1^1$ : Backlog items are set in progress and closed also on

days other than the sprint planning day and the sprint review day.

The distribution of user stories closed was explained by the developers' urge to demonstrate their completed user stories to their Product Owner as soon as it was completed, instead of waiting until the next sprint review day. According to a developer, after a user story was closed, integrated and the code successfully committed to the trunk of the source code repository, the code become "legacy" and the responsibility of managing commit conflicts and regression testing for that story was out of the team's hands. Thus the mid-sprint closing of user stories was also a way to reduce the number of commit conflicts in the source code repository, and to release the developer's responsibility for fixing conflicts. The mid-sprint closing of user stories allowed work to begin in any dependent user stories, which was not an uncommon occurrence due to the multi-team environment.

The distribution of stories set in progress was explained by the goal to keep the utilization of developers high. From the Product Owners' point of view, it did not make sense to wait till the next sprint planning day to give the developers new user stories to work on when they had completed their previous ones. This was especially true when the remaining user stories required specific skills that all team members did not have:

*...there may be things [user stories] which contain a massive task in a small [component] area where there may not be that much competence in the team. It may be a large item with a long lead time<sup>1</sup>, but nevertheless it doesn't contain work for the whole team in a sensible way. Then we have to take something else to the side.*

— A Product Owner (2013)

### C. User Story Development Time

The sprint length in the case organization is 14 days (i.e. two weeks). The descriptive statistics of the DIP of the user stories are as follows: the median  $\tilde{t} = 27$  and the mean  $\bar{t} \approx 31$  days. The one-sample Wilcoxon Signed-Rank Test confirms ( $p < 0.001$ ) that the mean DIP differs from the expected mean 14. We can also descriptively analyse the percent of user stories completed within different sprint counts. Table IV shows the number of stories ( $n$ ), percentage ( $s_i$ ), cumulative percentage (cum.  $s_i$ ) for each sprint count ( $i$ ). The value of  $s_1$  is only approximately 28%, i.e. less than a third of user stories were completed in a single sprint.

We must thus reject the null hypothesis  $H_0^2$  and accept the alternate hypothesis  $H_1^2$ :  $\tilde{t}$  and  $\bar{t}$  are significantly different from  $l$ . According to the interviews, there are two main reasons why user stories last for multiple sprints. The first reason is the inter-team dependencies and the dependencies to third parties:

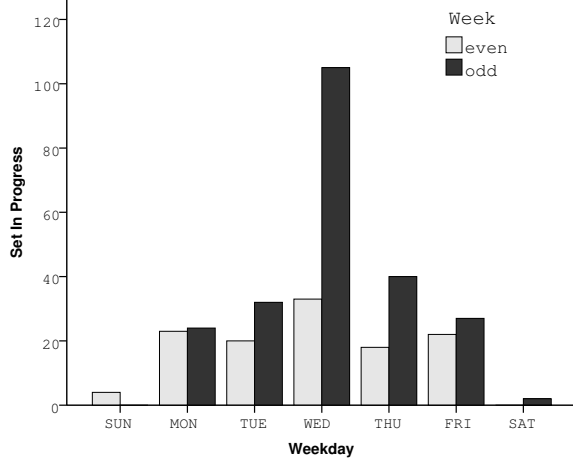
*...everything is not directly dependent on the team. Some things have a considerably longer lead time due to third parties. ...[one time] we had approximately one and a half months where almost nothing got closed ...Because we had certain external dependencies, I could not accept those things as closed before they were properly completed.*

— A Product Owner (2013)

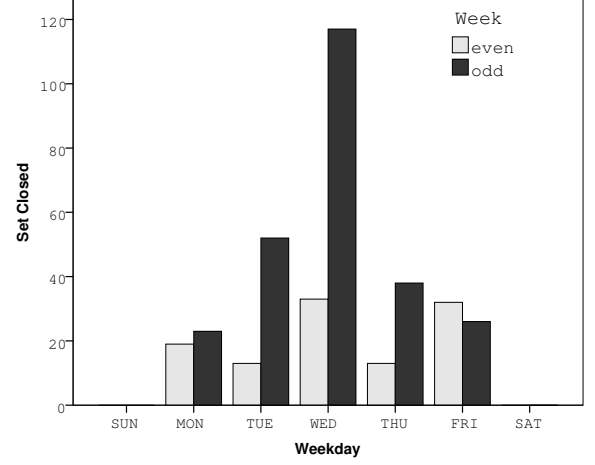
Table III  
USER STORIES AND DAYS OF DATA FOR EACH DEVELOPMENT TEAM

| Name   | User stories (#) | Data (days) |
|--------|------------------|-------------|
| Team A | 14               | 302         |
| Team B | 16               | 197         |
| Team C | 17               | 368         |
| Team D | 18               | 390         |
| Team E | 18               | 287         |
| Team F | 22               | 433         |
| Team G | 23               | 433         |
| Team H | 25               | 406         |
| Team I | 29               | 354         |
| Team J | 34               | 247         |
| Team K | 37               | 316         |
| Team L | 50               | 448         |

<sup>1</sup>Lead time refers to the time a user story is in progress.



(a) Set in progress



(b) Closed

Figure 3. Distribution of the User Stories (a) Set In Progress and (b) Closed Over Weekdays

The second reason is the difficulty of splitting user stories into pieces that can be implemented in a single sprint. The case organization aimed to create user stories that would create end-to-end functionality somehow visible to the customer or user. Due to the complex nature of the software system, creating such user stories that could be developed in a single sprint was perceived as challenging. The large DIP of user story implementation was not considered very harmful. However, there was a drive to decrease the size of user stories to increase the transparency of the software development:

*...the size of user stories is a compromise. In a way we try to make end-to-end things, something concrete and working in every story. And it creates certain demands for the size of the story. We would not like to go into having very small things in it ... If we have lots of these kinds of long-term things [user stories] ... the transparency suffers. Which causes anxiety in the managers because there is no visible progress.*

— A Product Owner (2013)

#### D. Estimates and Days In Progress

Since the teams in the case organization had employed common user stories as a means of comparison in estimation, we will first look at the overall relationship between the DIP and estimates for all included teams. We test whether there is a correlation by calculating Kendall's tau-b ( $\tau_B$ )

Table IV  
USER STORY COMPLETION PER NUMBER OF SPRINTS

| Sprints ( $i$ ) | Stories | $s_i$ (%) | cum. $s_i$ (%) |
|-----------------|---------|-----------|----------------|
| 1               | 99      | 28.4      | 28.4           |
| 2               | 88      | 25.3      | 53.7           |
| 3               | 69      | 19.8      | 73.6           |
| 4               | 30      | 8.6       | 82.2           |
| 5               | 26      | 7.5       | 89.7           |
| 6               | 21      | 6.0       | 95.7           |
| $\geq 7$        | 15      | 4.3       | 100.0          |

Table V  
KENDALL'S  $\tau_B$  BETWEEN THE DIP AND ESTIMATE

| Name          | $\tau_B$    | p (two-tailed) |
|---------------|-------------|----------------|
| <b>Team A</b> | <b>.576</b> | <b>.010</b>    |
| <b>Team B</b> | <b>.661</b> | <b>.002</b>    |
| Team C        | .351        | .070           |
| Team D        | .030        | .874           |
| <b>Team E</b> | <b>.484</b> | <b>.014</b>    |
| Team F        | .164        | .330           |
| Team G        | .291        | .072           |
| <b>Team H</b> | <b>.400</b> | <b>.013</b>    |
| Team I        | .223        | .132           |
| Team J        | .146        | .262           |
| <b>Team K</b> | <b>.277</b> | <b>.031</b>    |
| Team L        | .111        | .304           |

rank correlation coefficient.  $\tau_B = 0.255$  ( $p < 0.001$ ), which indicates that there is a very weak positive correlation between the DIP and estimate. To further analyse the correlation, we draw a scatter plot of DIP values over estimates, shown in Figure 4(a). The plot also shows the simple linear regression line (least squares estimator). The correlation is too weak to allow us to reject the null hypothesis  $H_0^3$  regarding all included development teams: The estimate and the time under development of backlog items do not correlate.

However, we cannot dismiss the possibility that the teams, regardless of the common baseline user stories, used different estimation scales during the study period. Thus, in addition, we test the hypothesis separate for each included team. Table V shows the Kendall's tau-b rank correlation coefficient and the associated p-value. The rows with teams that have statistically significant correlation are in bold. For Teams C, D, F, G, I, J and L we cannot reject the null hypothesis  $H_0^3$ , as the correlation is not significant.

Kendall's tau-b value describes how monotonic the correlation is, but it does not tell us anything else about the

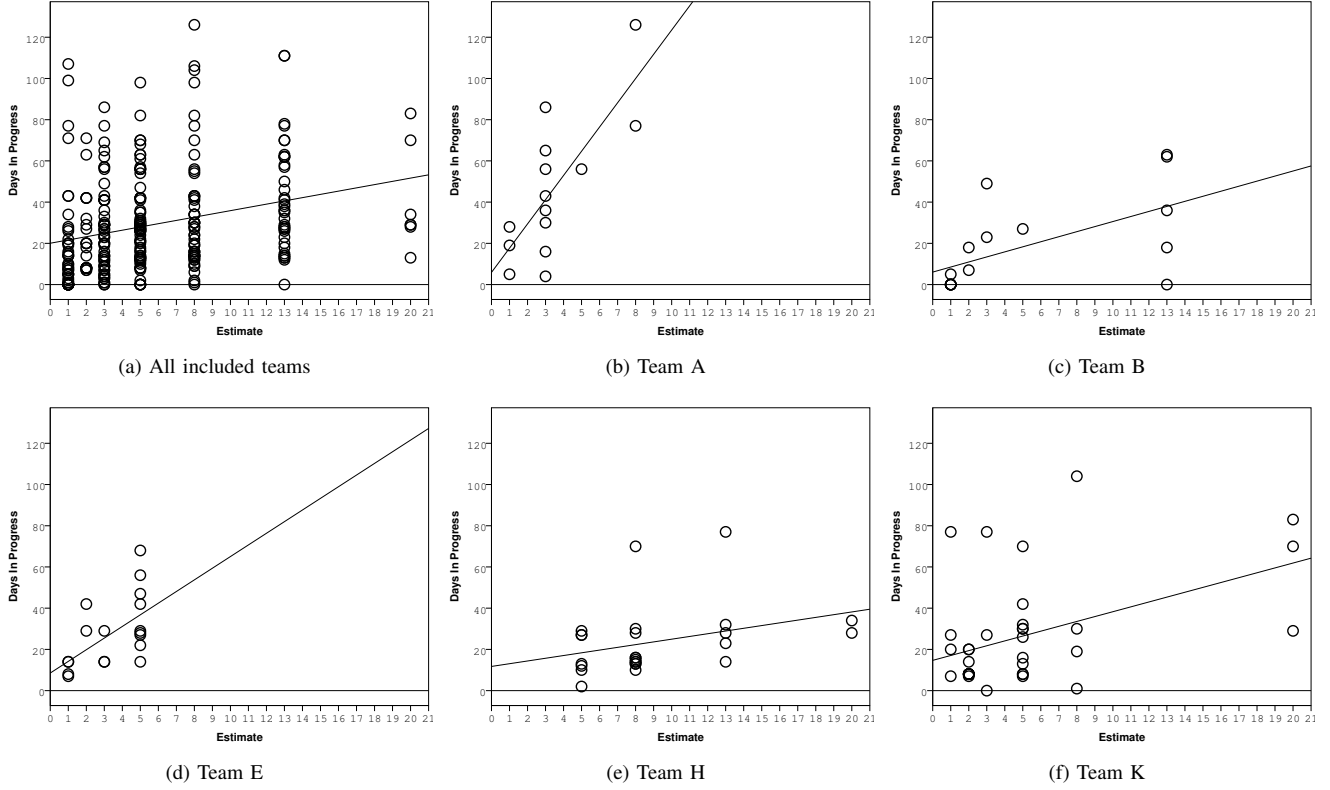


Figure 4. Scatter plots of the DIP over estimate (including a simple linear regression line)

relationship between the variables. To better understand the relationship between the DIP and the estimates of Teams A, B, E, H and K, we draw a scatter plot of the DIP over the estimate, which is shown in Figure 4(b-f). The plots also show the simple linear regression line (least squares estimator). A visual observation of the plots reveals differences in the relationship between the estimates and the DIP values. Teams B, H and K have a relatively similar, flat angle in the regression line, while Teams A and E have a much steeper angle. This suggests that the teams do use different estimation scales. In Scrum, we may expect a small increase in the DIP when the estimate increases, since a user story that was estimated too big will not get closed until the end of the sprint. However, a user story that was estimated too small may leak into the following sprint. This especially affects large user stories, as they have inherently more absolute estimate error than small user stories [10]. The plot of Team H may be explained by this phenomenon. Kendall's tau-b correlation of team K is very weak. Thus, we cannot reject the null hypothesis  $H_0^3$  regarding Teams H and K. For Team A, B and E, we can reject the null hypothesis and accept the alternative hypothesis  $H_1^3$ . The time under development and the estimates of backlog items have a positive correlation. Unfortunately, the interviews we conducted did not provide an explanation for these results.

## VI. DISCUSSION

In this section, we first answer our two research questions. Then we discuss the threats to the validity of the results.

### A. RQ1: How Were User Stories Planned and Managed in the Teams of the Case Organization?

The requirements management process of the case organization during the study period of 2011-2012 can be characterized as epic-centric. Concrete epics that provide customer-valuable functionality were split from the higher level abstract features created by the product manager. Depending on the size of an epic, one or more teams and Product Owners were allocated to develop the epic. During their bi-weekly backlog grooming meetings, the teams split user stories from the epics and estimated the user stories. The user stories were then set in progress in the following sprint planning meeting or during a sprint. When a user story was completed, tested and integrated, the team demonstrated the user story to the Product Owner who then officially closed it. The Product Owners were responsible for prioritizing user stories, while the Product Owners and the product manager prioritized and created epics together.

The organization of the software development was somewhat similar to the feature team model [14], which is a two-layer model for scaling a large-scale development organization. In the model, the development teams are



arranged as feature teams that work on a single feature at a time. Feature teams are grouped into technical product areas. Each product area is managed by an Area Product Owner, who in turn are managed by a (Chief) Product Owner. The Product Owner manages the product backlog and assigns backlog items to the product areas. Features are large backlog items that describe functionality that is valuable for the customer. Features are split into smaller backlog items which can be implemented during a single sprint.

The main differences between the feature team model [14] and our case were that in the feature team model each feature is allocated to a single team, and the teams are grouped by the affected technical product areas that each have a responsible area Product Owner. In the case organization the size of the epics varied considerably (at least by a factor of 100) and the varying number of teams assigned to develop each epic was employed to keep the development time of large epics within reasonable limits and to increase the flexibility of the development organization. In addition, the knowledge of the product spread amongst the Product Owners, which increased the flexibility of the development organization and mitigated personnel risks further.

*B. RQ2: Did the User Story Planning and Management Practices in the Teams Differ From the Scrum Practices, and If They Did, How Were the Differences Explained?*

We identified three Scrum practices and compared the user story metrics from the case organization to them. The first Scrum practice was the sprint planning days when the user stories are set in progress, and the sprint review days when they are closed. According to Scrum, the great majority of user stories should be set in progress during the sprint planning day and closed during the sprint review day. The results from the case organization regarding these metrics were mixed. Only 30% of user stories were set in progress and 32% of user stories were closed during the sprint planning day and the sprint review day, respectively. The results from the case organization do not match Scrum regarding the first practice.

The mismatch with the first Scrum practice was explained by the Product Owners' attempt to keep the utilization of the developers high, and by the developers desire to get user stories closed as soon as they were completed. When user stories become completed during a sprint, the developers demonstrated the user story to the Product Owner. If the sprint contained no additional user stories to which the developers could efficiently contribute to, the Product Owner took in a new user story. This was typically caused by user stories that demanded specific skills that not everyone in the team had. This mismatch with Scrum was seen as an improvement over Scrum, as the utilization of developers and the efficiency of the development teams could be kept on a high level.

The second Scrum practice was the user story development pace. According to Scrum, the great majority of user stories should be in progress exactly for one sprint. In Scrum, user stories must be small enough to be completed in a single sprint and the sprint velocity limits the number of user stories

that can be included in a single sprint. This was clearly not the case in the case organization. Only approximately 28% of the user stories were completed in a single sprint's time and only approximately 74% of the user stories were completed in three sprints or less. The results from the case organization do not match Scrum regarding the second practice.

The mismatch with the second Scrum practice was explained by the complex nature of the product, and by the inter-team and external dependencies of the user stories. Some user stories focused on small component areas that required high skills and experience. Since all team members did not have high skills and experience in all component areas, the number of people concurrently developing such user stories was limited. As a result, such user stories could take multiple sprints to be completed. Some user stories had inter-team or external dependencies which prevented the user story from being closed although most of the work regarding the user story was completed on the development team's side. The long in progress time of user story implementation was not considered too problematic, as the release projects were three months (or six sprints) long and the contents of each release were based on what was completed in time. However, long-running user stories decreased the visibility of the development progress to the Product Owners, which was seen as a problem. Overall, this mismatch with Scrum was perceived slightly problematic, but unavoidable in the context of the case organization.

The third Scrum practice is the use of sprint velocity and estimates to limit the amount of work included in each sprint. According to Scrum, the user story development time should be independent of the estimate, as all included user stories should be completed during a single sprint regardless of the estimate. As described above, most user stories in the case organization were in progress more than one sprint. Large user stories should be split into smaller ones if they cannot be fit into a single sprint. No notable correlation between the DIP and estimate was found in the analysis of all included development teams. Due to the possible differences in estimation scales between the teams in the case organization, we also analysed the teams separately. Based on the team-level analysis, we could not say whether the estimates of user stories correlate with the DIP for most of the teams. However, we identified several teams that had a significant positive correlation. The results from these teams do not match with Scrum regarding the third practice. However, we were unable to qualitatively explain these results.

The changes to the Scrum practices in the case organization were similar to the practices proposed in the Kanban software development method [9]. In Kanban, the development process is a continuous process where new user stories are taken into development when previous ones are completed. The amount of concurrent work is limited by a work-in-progress (WIP) measure. The user stories in Kanban do not have an explicit in progress time limit, but the goal is to optimize the development flow by balancing the development lead

time and the utilization level of development resources. However, to our knowledge, the development teams in the case organization did not employ a WIP limit and did not explicitly monitor the development lead time. In addition, their goal was to maximize the development utilization instead of minimizing the development lead time.

### C. Threats to Validity and Generalizability

This study was based on the qualitative and quantitative data from a single organization. We cannot make generalizations to other organization based on these results. The qualitative analysis of the interviews was performed by the authors together, which limited the effects of possible researcher bias in the analysis and increased the construct validity [12], [15].

The quantitative data was produced by the case organization. We cannot directly ascertain that the quantitative backlog data fairly represents the reality in the case organization. However, the results from the 39 interviews conducted in 2011 support the results from the quantitative analysis. Furthermore, the quantitative results were validated by the two experienced and authoritative members of the case organization during the interview we conducted in 2013. This triangulation of quantitative and qualitative data increases the construct validity of the results [12], [15].

## VII. CONCLUSIONS AND FUTURE WORK

Scrum sets certain limits to the development time and pace of user stories. Organizations that do not follow these limits are said to suffer from ScrumButs which hide organizational problems revealed by Scrum by altering the core practices of Scrum. We first described quantitatively and qualitatively what the limits imposed by Scrum are, and then studied whether these limits were observed in a case organization. We identified two mismatches with Scrum. The first of these was the tendency to set in progress and close user stories during a sprint. The second was the multi-sprint in progress time of majority of user stories. Only the second deviation was considered slightly harmful. Our results suggest that changes to the Scrum user story management and planning process cannot be said to be categorically harmful ScrumButs.

In the future, we continue to study this organization. We will study qualitatively how the individual teams in the case organization perform sprint planning and backlog management. In this study, we identified differences between the teams in the correlation between estimates and days in progress of the user stories, but we were unable to explain the sources of those differences. By qualitatively studying those teams, we aim to discover the causes of the differences. We will also study how the business performance

of this organization has changed over time and correlate it with the changes made to the software development process to estimate the effects of different process changes to the business performance. In addition, we aim to perform similar studies in other companies to investigate the generalizability of the phenomenon discovered in this study.

## ACKNOWLEDGMENT

This research has been funded by the TiViT Cloud Software Finland program.

## REFERENCES

- [1] K. Schwaber and M. Beedle, *Agile software development with Scrum*. Upper Saddle River, NJ: Prentice-Hall, 2002.
- [2] K. Schwaber and J. Sutherland, "The Scrum guide - the definitive guide to Scrum: The rules of the game," Scrum.org, Tech. Rep., October 2011. [Online]. Available: <http://www.scrum.org/Scrum-Guides>
- [3] Scrum.org. (2013, March) ScrumButs and modifying Scrum. [Online]. Available: <http://www.scrum.org/ScrumBut>
- [4] K. Schwaber, *Agile project management with Scrum*. Redmond, WA: Microsoft Press, 2004.
- [5] V. Heikkilä, K. Rautiainen, and S. Jansen, "A revelatory case study on scaling agile release planning," in *Proceedings of the 36th Euromicro Conference on Software Engineering and Advanced Applications*. IEEE Computer Society, 2010, pp. 289–296.
- [6] D. Leffingwell, *Agile software requirements : lean requirements practices for teams, programs, and the enterprise*. Upper Saddle River, NJ: Addison-Wesley, 2011.
- [7] V. T. Heikkilä, M. Paasivaara, C. Lassenius, and C. Engblom, "Continuous release planning in a large-scale Scrum development organization at Ericsson," in *Agile Processes in Software Engineering and Extreme Programming*, ser. Lecture Notes in Business Information Processing, H. Baumeister and B. Weber, Eds., vol. 149. Springer Berlin Heidelberg, 2013, pp. 195–209.
- [8] D. I. K. Sjøberg, A. Johnsen, and J. Solberg, "Quantifying the effect of using Kanban versus Scrum: A case study," *Software, IEEE*, vol. 29, no. 5, pp. 47–53, 2012.
- [9] H. Kniberg, *Lean from the Trenches: Managing Large-Scale Projects with Kanban*. Pragmatic Bookshelf, 2011.
- [10] M. Cohn, *User stories applied : for agile software development*. Boston: Addison-Wesley, 2004.
- [11] —, *Agile estimating and planning*. Upper Saddle River, NJ: Prentice Hall Professional Technical Reference, 2005.
- [12] M. Q. Patton, *Qualitative research and evaluation methods*, 3rd ed. Thousand Oaks, CA: Sage Publications, 2002.
- [13] H. Coolican, *Research methods and statistics in psychology*, 3rd ed. London: Hodder & Stoughton, 1999.
- [14] C. Larman and B. Vodde, *Practices for scaling lean & agile development: large, multisite, and offshore product development with large-scale scrum*. Upper Saddle River, NJ: Addison-Wesley, 2010.
- [15] R. K. Yin, *Case study research : design and methods*, 4th ed. Thousand Oaks, CA: Sage Publications, 2009.